

SECTION: PRINCIPLES

Principles

Single Responsibility Principle 🚧 ⚡ ⚡ ⚡

📖 **Single responsibility principle (SRP):** A class should have one, and only one, reason to change. -- Robert C. Martin

If a class has only one responsibility, it needs to change only when there is a change to that responsibility.

📦 Consider a `TextUi` class that does parsing of the user commands as well as interacting with the user. That class needs to change when the formatting of the UI changes as well as when the syntax of the user command changes. Hence, such a class does not follow the SRP.

“Gather together the things that change for the same reasons. Separate those things that change for different reasons.” — *Agile Software Development, Principles, Patterns, and Practices* by Robert C. Martin

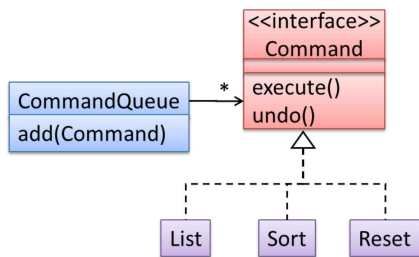
Open-Closed Principle 🚧 ⚡ ⚡ ⚡

The Open-Closed Principle aims to make a code entity easy to adapt and reuse without needing to modify the code entity itself.

📖 **Open-closed principle (OCP):** A module should be *open* for extension but *closed* for modification. That is, modules should be written so that they can be extended, without requiring them to be modified. -- proposed by [Bertrand Meyer](#)

In object-oriented programming, OCP can be achieved in various ways. This often requires separating the *specification* (i.e. *interface*) of a module from its *implementation*.

📦 In the design given below, the behavior of the `CommandQueue` class can be altered by adding more concrete `Command` subclasses. For example, by including a `Delete` class alongside `List`, `Sort`, and `Reset`, the `CommandQueue` can now perform delete commands without modifying its code at all. That is, its behavior was extended without having to modify its code. Hence, it is open to extensions, but closed to modification.



📦 The behavior of a Java generic class can be altered by passing it a different class as a parameter. In the code below, the `ArrayList` class behaves as a container of `Students` in one instance and as a container of `Admin` objects in the other instance, without having to change its code. That is, the behavior of the `ArrayList` class is extended without modifying its code.

```
1 ArrayList students = new ArrayList<Student>();
2 ArrayList admins = new ArrayList<Admin>();
```

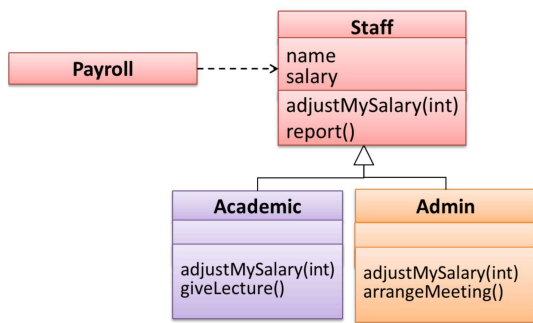
Liskov Substitution Principle 🚧 ⚡ ⚡ ⚡



Liskov substitution principle (LSP): Derived classes must be substitutable for their base classes. -- proposed by [Barbara Liskov](#)

LSP sounds the same as substitutability but it goes beyond substitutability; **LSP implies that a subclass should not be more restrictive than the behavior specified by the superclass.** As you know, Java has language support for substitutability. However, if LSP is not followed, substituting a subclass object for a superclass object can break the functionality of the code.

Suppose the `Payroll` class depends on the `adjustMySalary(int percent)` method of the `Staff` class. Furthermore, the `Staff` class states that the `adjustMySalary` method will work for all positive percent values. Both the `Admin` and `Academic` classes override the `adjustMySalary` method.



Now consider the following:

- The `Admin#adjustMySalary` method works for both negative and positive percent values.
- The `Academic#adjustMySalary` method works for percent values `1..100` only.

In the above scenario,

- The `Admin` class follows LSP because it fulfills `Payroll`'s expectation of `Staff` objects (i.e. it works for all positive values). Substituting `Admin` objects for `Staff` objects will not break the `Payroll` class functionality.
- The `Academic` class violates LSP because it will not work for percent values over `100` as expected by the `Payroll` class. Substituting `Academic` objects for `Staff` objects can potentially break the `Payroll` class functionality.

➤ Another example

SOLID Principles 🔥🔥🔥

The five OOP principles given below are known as *SOLID Principles* (an acronym made up of the first letter of each principle):

➤ **S**ingle Responsibility Principle (SRP)

➤ **O**pen-Closed Principle (OCP)

➤ **L**iskov Substitution Principle (LSP)

➤ **I**nterface Segregation Principle (ISP)

Separation of Concerns Principle ⚡⚡

 **Separation of concerns principle (SoC):** To achieve better modularity, separate the code into distinct sections, such that each section addresses a separate *concern*. -- Proposed by [Edsger W. Dijkstra](#)

A *concern* in this context is a set of information that affects the code of a computer program.

 Examples for *concerns*:

- A specific feature, such as the code related to the `add employee` feature
- A specific aspect, such as the code related to `persistence` or `security`
- A specific entity, such as the code related to the `Employee` entity

Applying SoC reduces functional overlaps among code sections and also limits the ripple effect when changes are introduced to a specific part of the system.

 If the code related to *persistence* is separated from the code related to *security*, a change to how the data are persisted will not need changes to how the security is implemented.

This principle can be applied at the class level, as well as at higher levels.

 The n-tier architecture utilizes this principle. Each layer in the architecture has a well-defined functionality that has no functional overlap with each other.

This principle should lead to higher cohesion and lower coupling.

Law of Demeter ⚡⚡⚡

 **Law of Demeter (LoD):**

- An object should have limited knowledge of another object.
- An object should only interact with objects that are closely related to it.

Also known as

- Don't talk to strangers.
- Principle of least knowledge

More concretely, a method `m` of an object `o` should invoke only the methods of the following kinds of objects:

- The object `o` itself
- Objects passed as parameters of `m`
- Objects created/instantiated in `m` (directly or indirectly)
- Objects from the direct association of `o`.

 The following code fragment violates LoD due to the following reason: while `b` is a 'friend' of `foo` (because it receives it as a parameter), `g` is a 'friend of a friend' (which should be considered a 'stranger'), and `g.doSomething()` is analogous to 'talking to a

stranger’.

```
1 void foo(Bar b) {  
2     Goo g = b.getGoo();  
3     g.doSomething();  
4 }
```

LoD aims to prevent objects from navigating the internal structures of other objects.

📦 An analogy for LoD can be drawn from Facebook. If Facebook followed LoD, you would not be allowed to see posts of friends of friends, unless they are your friends as well. If Jake is your friend and Adam is Jake’s friend, you should not be allowed to see Adam’s posts unless Adam is a friend of yours as well.